FINAL REPORT

Intern Technical Assignment

Finance Management System

HSClogic Pvt Ltd

Date: 24 April 2026

Version: 1.0 - Final

Prepared by: KMCP Munasinghe

System: HSCLogic Finance Management System

Confidential

Table of Contents

Table of Contents 2
1. Executive Summary 4
2. Introduction 5
2.1 Background 5
2.2 Project Objectives 5
2.3 Scope of the Report 5
2.4 References 5
3. System Architecture 6
3.1 Application Architecture 6
3.2 State Management Pattern 6
3.3 Page and Feature Structure 6
4. Technology Stack 7
4.1 Technology Justifications 7
5. Functional Modules 8
5.1 Authentication 8
5.2 Dashboard 8
5.3 Invoice Management 8
5.4 Employee Management 9
5.5 Salary Structures 9
5.6 Payroll Processing 10
6. Role-Based Access Control 11
6.1 Permission Matrix 11
6.2 Enforcement Architecture 12
7. User Interface Design 12
7.1 Design System 12
7.2 Responsive Design Strategy 12
7.3 Demo Accounts 13
8. PDF Generation 13
8.1 Invoice PDF 13
8.2 Payslip PDF 13
8.3 Payroll Report PDF 13

8.4 Salary Structure Report PDF 14
9. Data Management 14
9.1 Context Providers 14
9.2 Mock Data 14
9.3 Automatic Overdue Detection 14
9.4 Salary Snapshot on Payroll Run 14
10. Key Business Rules and Validations 15
10.1 Invoice Validations 15
10.2 Employee Validations 15
10.3 Salary Validations 15
10.4 Payroll Business Rules 15
11. Project Structure 15
12. Setup and Running the Project 16
12.1 Prerequisites 16
12.2 Installation and Development 16
12.3 Production Build 16
13. Known Limitations and Future Enhancements 17
13.1 Current Limitations 17
13.2 Recommended Future Enhancements 17
14. Conclusion 18

1. Executive Summary

This report presents the design, development and implementation of the Finance Management System (FMS) developed for HSClogic Pvt Ltd as a pre-internship interview technical assignment. HSClogic is a technology and consultancy company offering software development and 3D printing services. The assignment required developing the frontend of a web-based financial management platform within a two-week timeframe, based on the Software Requirements Specification (SRS) and Functional Requirements Specification (FRS) provided by HSClogic. The FMS was built to centralize and automate core financial operations including invoice management, employee records, salary structure definition, and monthly payroll processing.

As specified in the SRS constraints, the initial implementation focuses on the frontend/UI layer. The system is a frontend only single page application (SPA) built entirely with React 19, Vite 8, and Tailwind CSS 3. There is no backend server or database in the current implementation - all state is managed in-memory using React Context API and seeded from static mock data files. localStorage is used to persist in the authenticated user session across page reloads. All monetary values are denominated in Sri Lankan Rupees (LKR). Backend integration with a RESTful API can be added in future phases as outlined in the SRS.

The FMS supports three user roles - Admin, Finance Manager and Employee, with strict role-based access control enforced at both the route level and the UI level, as defined in the SRS user classes and FRS user roles. The system generates professional, branded PDF documents (invoices, pay slips, payroll reports, salary reports) entirely on the client side using jsPDF, with no server round-trip required. The application is fully responsive from 375px mobile screens to full desktop displays, meeting the FRS UI functional expectations for clean, responsive design with validation messages and clear table displays.

2. Introduction

2.1 Background

HSClogic Pvt Ltd is a technology and consultancy company specializing in software development and 3D printing services. As part of the internship interview process, HSClogic provided a technical assignment requiring candidates to develop the frontend of a Finance Management System. The assignment was accompanied by two reference documents - a Software Requirements Specification (SRS) and a Functional Requirements Specification (FRS), both dated 11 April 2026 - which defined the functional and non-functional requirements, user roles, module specifications and acceptance criteria. The development timeline was set at two weeks, with the scope explicitly limited to the frontend/UI implementation phase.

2.2 Project Objectives

Based on the SRS and FRS module specifications, the primary objectives of the Finance Management System were:

- Provide a single platform for managing invoices, employees, salary structures, and payroll
- Enforce strict role-based access so each user type (Admin, Finance Manager, Employee) sees only what is relevant to them
- Automate payroll calculation from salary structures to reduce manual arithmetic errors
- Generate professional, branded PDF documents (invoices, payroll reports, salary reports) at the click of a button
- Deliver a fully responsive UI that works on mobile phones (375px and above) as well as desktop browsers
- Automatically detect and flag overdue invoices without requiring manual intervention

2.3 Scope of the Report

This report covers the full scope of the Finance Management System as implemented for the intern technical assignment - from its architecture and technology choices through each functional module, the role-based access control model, user interface design, PDF generation capabilities, data management approach, business rules and validations, project structure, setup instructions, and known limitations with recommended future enhancements. As stated in the SRS constraints, the focus of this implementation phase is the frontend/UI layer, with backend integration planned for future phases. The report demonstrates how each SRS functional requirement and FRS acceptance criterion have been addressed.

2.4 References

- Software Requirements Specification (SRS) - Finance Management System, HSClogic, 11 April 2026
- Functional Requirements Specification (FRS) - Finance Management System, HSClogic, 11 April 2026

- Intern Technical Assignment Brief - provided by HSClogic

3. System Architecture

HSCLogic FMS is a single-page application (SPA) built entirely on the frontend. There is no backend server or database in the current implementation; all state is managed in-memory using React Context API and seeded from static mock data files. localStorage is used to persist in the authenticated user session across page reloads.

3.1 Application Architecture

The application follows component-based architecture with clear separation of concerns. React Router v7 provides client-side routes with protected and role-guarded routes. Five React Context stores manage the application state: AuthContext (user session, login/logout, role permission checks), InvoiceContext (invoice CRUD and live overdue detection via useMemo), EmployeeContext (employee CRUD and salary structure management), PayrollContext (payroll processing, finalisation, deletion), and ThemeContext (light/dark mode toggle).

3.2 State Management Pattern

Each domain (invoices, employees, payroll) has its own Context Provider that encapsulates the initial seeded state, all CRUD operations exposed to consumer components, and derived computed state. For example, InvoiceContext uses useMemo to automatically detect and flag overdue invoices on every render cycle, requiring no manual intervention.

3.3 Page and Feature Structure

Feature pages include DashboardPage (role-branched: management view vs employee personal view), InvoicesPage (full invoice lifecycle management), EmployeesPage (employee directory and profile management), SalaryPage (salary structure editor and report), PayrollPage (monthly payroll processing and payslip viewer), and LoginPage (credential-based authentication). Shared UI components (Card, Modal, Badge, Button, Input, ConfirmDialog, Layout, Sidebar, Topbar) provide visual consistency across the application.

4. Technology Stack

The following technologies were used to build the Finance Management System. The FRS specified React or Angular as the frontend framework, with additional considerations for feature-based architecture, responsive design, form validation, and state management. React was chosen. All dependencies are frontend only - no server-side runtime or database is required, as the assignment scope is limited to the UI implementation phase per SRS.

Category	Technology	Version	Purpose
UI Framework	React	19.2.4	Component-based user interface
Build Tool	Vite	8.0.4	ES module-native bundling and HMR
Routing	React Router DOM	7.14.1	Client-side routing with role guards
Styling	Tailwind CSS	3.4.19	Utility-first CSS with theme support
Charts	Recharts	3.8.1	SVG-based interactive charts
PDF Generation	jsPDF	4.2.1	Client-side PDF document creation
HTML-to-Canvas	html2canvas	1.4.1	DOM element capture for PDF
Icons	Lucide React	1.8.0	Icon library
Notifications	React Hot Toast	2.6.0	Toast notification system
Form Utilities	React Hook Form	7.72.1	Form state management
Language	JavaScript (ESM)	ES2022	Application source language
Linting	ESLint	9.39.4	Code quality enforcement

4.1 Technology Justifications

React 19 with Vite 8 was chosen for its fast local development server, ES module-native bundling, and minimal configuration overhead. Vite's hot module replacement enables near-instant UI iteration during development.

Tailwind CSS 3 provides utility-first classes combined with CSS custom property theming, enabling full light/dark mode support without any JavaScript overhead or additional stylesheets.

React Router 7 enables nested route layouts, allowing the sidebar and topbar shell to wrap all authenticated pages through a single Layout route, eliminating duplication. jsPDF runs entirely in the browser with no server round-trip required, producing all four PDF document types of client-side.

5. Functional Modules

The Finance Management System comprises five core functional modules, addressing the two primary functional areas defined in the SRS: Invoice Management and Salary & Payroll Management. The implementation adds a Dashboard module, an authentication module, and extends beyond the minimum requirements with features such as automatic overdue detection, invoice duplication, bulk PDF export, and dual-view employee browsing. Each module shares a common authentication framework through the Context API.

5.1 Authentication

The authentication module provides credential-based login validated against a mock user store. Key features include:

- Email and password login with inline error display
- Session persisted with localStorage to survive page refresh
- Automatic redirect to /login for unauthenticated route access
- Automatic redirect to /dashboard if already authenticated and visiting /login
- Three pre-configured demo accounts (Admin, Finance Manager, Employee) with auto-fill buttons on the login page

5.2 Dashboard

The dashboard renders a completely different view depending on the logged-in user's role.

Admin and Finance Manager view: Displays total revenue from paid invoices, invoice summary (total, paid, pending, overdue counts), overdue count with contextual attention indicator, active employee count, total salary disbursed across finalized payrolls, employees included in latest payroll, pending payroll count, Revenue vs Payroll area chart (last 6 months), invoice status distribution pie chart, recent invoices table (latest 5), and latest payroll card with per-employee net pay breakdown.

Employee view: Displays a personal profile card (name, designation, department, employee ID, join date), four summary stats (latest net pay, gross earnings, deductions, total earned), earnings history area chart (net vs gross for last 6 months), latest payslip card with fully itemised breakdown, and complete payroll history table.

5.3 Invoice Management

The invoice management module implements the full invoice lifecycle as specified in SRS and FRS. All SRS requirements - CRUD operations, auto-calculation, status tracking (Draft/Sent/Paid/Overdue), and PDF document generation - are implemented. Key capabilities include:

- Create invoices with client name, service type (Software / 3D Printing / Others), issue date, due date, line items (description, quantity, unit price), and notes
- Auto-generated sequential invoice numbering (e.g. INV-001)
- Real-time auto-calculation of line totals and invoice totals
- Status lifecycle: Draft → Sent → Paid → Overdue
- Automatic overdue detection - invoices past due date are flagged automatically via useMemo derived state, requiring no manual step
- Inline status change via popover on the status badge
- Invoice duplication to create a new Draft pre-populated with same line items
- Bulk PDF export of all currently filtered invoices
- Search by client name or invoice number, and filter by status (All / Draft / Sent / Paid / Overdue)
- Finance Manager access: can view and export but cannot create, edit, duplicate, delete, or change status

5.4 Employee Management

The employee management module maintains the organisation's employee directory. Features include:

- Full employee record: name, auto-generated employee ID, designation, department, date of joining, email, phone, and status
- Activate and deactivate employees (soft deletion retaining all records)
- Search by name, employee ID, or designation
- Filter by status (All / Active / Inactive)
- Dual view modes: table view for dense data scanning, card grid view for profile-style browsing
- CSV export of the current filtered employee list
- Admin-only actions: Add Employee, Edit Employee, Activate/Deactivate

5.5 Salary Structures

The salary structure module implements per-employee salary configuration as defined in SRS and FRS. The salary formula is: Net Salary = Basic Salary + Sum(Allowances) - Sum(Deductions). Key features include:

- Per-employee salary structure: basic salary + multiple named allowances + multiple named deductions
- Allowances and deductions are individually named (e.g. Transport Allowance, Meal Allowance, EPF 8%, ETF 3%)
- Salary changes do not retroactively affect processed payroll records - each payroll run takes a snapshot at the time of processing

- Active employee detail cards show the full breakdown with individual component values
- Salary structure report PDF exportable as a full summary of all employees

5.6 Payroll Processing

The payroll processing module implements monthly payroll runs as specified in SRS and FRS

. The SRS constraint that finalised payroll records cannot be modified is enforced. Key capabilities include:

- Monthly payroll run: Admin selects month and year, system processes all active employees using current salary structures
- Duplicate prevention: system blocks payroll runs for month/year combinations that already have a record
- Payroll status lifecycle: Pending → Finalised
- Finalisation permanently locks the payroll record - cannot be edited or deleted
- Per-payroll detail modal showing employee-level breakdown of basic, allowances, deductions, and net pay
- Payroll report PDF covering all employees with a total payout summary row
- Individual payslip PDF per employee with fully itemised earnings and deductions
- Payroll history filterable by year
- Employee role restriction: employees can only see finalised payrolls containing their own record
- Admin can delete Pending payroll records; Finalised records are protected

6. Role-Based Access Control

The system enforces strict role-based access control through the three user roles defined in SRS and FRS: Admin (full access), Finance Manager (invoices + payroll viewing), and Employee (view-only for own data). Access is enforced at two independent layers to ensure security even when URLs are typed directly, meeting the SRS security requirement that users shall only access authorized functionalities.

6.1 Permission Matrix

The following table summarizes the permissions assigned to each role across all system features.

Feature	Admin	Finance Mgr	Employee
Dashboard - management view	Yes	Yes	No
Dashboard - personal view	No	No	Yes
View Invoices	Yes	Yes	No
Create / Edit / Delete Invoices	Yes	Yes	No
Change Invoice Status	Yes	Yes	No
Export Invoice PDF	Yes	Yes	No
View All Employees	Yes	No	No
Add / Edit / Deactivate Employees	Yes	No	No
View / Edit Salary Structures	Yes	No	No
Export Salary Report PDF	Yes	No	No
View All Payroll Records	Yes	Yes	No
View Own Payroll Records Only	Yes	Yes	Yes
Process / Finalize Payroll	Yes	No	No
Delete Pending Payroll	Yes	No	No
Download Own Pay slip PDF	Yes	Yes	Yes
Download Payroll Report PDF	Yes	Yes	No

6.2 Enforcement Architecture

Route level: The RoleRoute component in AppRouter.jsx wraps each sensitive route. It checks user.role against the allowed roles array and issues a redirect to /dashboard for any unauthorised access attempt, regardless of how the user reached the URL.

UI level: The can(action) function from AuthContext returns a boolean based on a permission map for each role. Individual buttons, form submissions, action menus, and data sections are wrapped in conditional rendering so that write actions are never exposed to unauthorised roles.

7. User Interface Design

7.1 Design System

The UI meets the FRS functional UI expectations for forms with clear validation messages, clean table displays, CRUD action buttons, responsive layout, and professional presentation. It uses a custom Tailwind CSS theme built on CSS custom properties, enabling seamless light and dark mode switching with a single class toggle on the root HTML element. Key design tokens include: --bg (page background), --surface (card/panel background), --border (all borders), --text (primary text), --green (brand accent / success), and --red (error / danger / deductions). The theme toggle is available in the topbar when authenticated and on the login page.

7.2 Responsive Design Strategy

The application is fully responsive from 375px (iPhone SE) and wider. The sidebar navigation is a fixed 224px drawer on desktop that slides in from off-screen on mobile with a backdrop overlay. Modal dialogs appear centered on desktop and slide up as bottom sheets on mobile. Data tables are wrapped in overflow-x-auto containers with progressive columns hiding using Tailwind responsive classes. Stat card grids use grid-cols-2 on mobile and grid-cols-4 on desktop.

7.3 Demo Accounts

Three pre-configured accounts are available on the login page with auto-fill buttons for quick access:

Role	Email	Password	Access Summary
Admin	admin@hsclogic.com	admin123	Full access to all modules
Finance Manager	dilani.r@hsclogic.com	finance123	Invoices (full CRUD), payroll viewing, own payslips
Employee	ashan.perera@hsclogic.com	emp123	Personal dashboard only, own payslips

8. PDF Generation

All PDFs are generated entirely in the browser using jsPDF. No server is involved. Four document types are produced:

8.1 Invoice PDF

Filename format: INV-XXXX.pdf. Contents include HSCLogic branding with green accent header strip, client name and service type, invoice number and status, issue date and due date, line items table with quantity and unit price columns, auto-calculated line totals, subtotal row, tax row (0%), and total due highlighted in a green summary box. Notes section is appended if present. Footer includes system name and generation date.

8.2 Payslip PDF

Filename format: Payslip-EMP-XXX-Month-YYYY.pdf. Contents include employee name, designation, department, employee ID, and pay period header. An earnings section lists basic salary and each named allowance individually with a gross earnings subtotal. A deductions section lists each named deduction in red with a total deductions subtotal. Net salary is displayed in a branded green highlight box showing the calculation formula (Gross − Deductions = Net).

8.3 Payroll Report PDF

Filename format: Payroll-Month-YYYY.pdf. Contents include period, employee count, status, processed date, and total payout in a metadata block. Per-employee table rows show basic, allowances (green), deductions (red in parentheses), and net pay. A total payout appears in a green summary footer row.

8.4 Salary Structure Report PDF

Filename format: Salary-Report-YYYY-MM-DD.pdf. Contents include active employee count and total monthly payroll in a summary header. A per-employee table shows department, basic, allowances (green), deductions (red), and net pay. Total monthly payroll appears in a green footer row. The report supports automatic page breaks for large employee lists.

9. Data Management

9.1 Context Providers

Five React Context providers manage all application state:

- AuthContext (AuthContext.jsx): User session, login, logout, role-based permission checks via can(action)
- InvoiceContext (InvoiceContext.jsx): Invoice CRUD, auto-overdue detection via useMemo derived state
- EmployeeContext (EmployeeContext.jsx): Employee CRUD, salary structure updates, net salary calculation
- PayrollContext (PayrollContext.jsx): Payroll processing, status finalisation, deletion
- ThemeContext (ThemeContext.jsx): Light/dark mode toggle with localStorage persistence

9.2 Mock Data

The system is seeded with static mock data files that provide realistic initial state:

- mockInvoices.js: 10 seeded invoices across multiple clients (TechVision Lanka, EduSpark Institute, Bright Minds Academy, Robonautics Lab), service types, and statuses
- mockEmployees.js: 4 employees with full profiles and salary structures - Ashan Perera (Senior Software Engineer, Engineering), Nimasha Fernando (UI/UX Designer, Design), Kasun Jayawardena (3D Print Technician, Production), Dilani Rathnayake (Finance Manager, Finance)
- mockPayroll.js: 3 finalised payroll runs (January–March 2026) with per-employee records including named allowances (Transport, Meal, Professional) and deductions (EPF 8%, ETF 3%)

9.3 Automatic Overdue Detection

Invoice overdue status is not stored as a static value. InvoiceContext uses a useMemo derived state that re-evaluates on every render cycle. The isOverdue function returns true when the due date has passed and the invoice status is not Paid. This means overdue status updates automatically as time passes with no manual intervention required.

9.4 Salary Snapshot on Payroll Run

When payroll is processed, the system reads each active employee's current salary structure and stores a complete snapshot (basic, named allowances, named deductions, computed net) in the

payroll record. Subsequent edits to salary structures do not alter historical payroll records, preserving audit integrity for past periods.

10. Key Business Rules and Validations

10.1 Invoice Validations

- Client name is required and must not be blank
- Issue date is required
- Due date is required and must be on or after the issue date
- Each line item must have a non-empty description
- Quantity per line item must be greater than zero
- Unit price per line item must be zero or greater

10.2 Employee Validations

- Full name, designation, department, date of joining, email, and phone are all required
- Email must match standard format (user@domain.tld)

10.3 Salary Validations

- Basic salary must be greater than zero
- All allowance and deduction entries must have names and non-negative amounts

10.4 Payroll Business Rules

- Cannot process payroll if there are no active employees
- Cannot run payroll for a month/year combination that already has an existing record
- Finalised payroll records cannot be edited or deleted under any circumstances
- Pending payroll records can be deleted by Admin only
- Employees can only view and download their own payslips - no cross-employee data is exposed

11. Project Structure

The project follows a feature-based directory structure under the src/ folder. Key directories include:

- src/assets/ - Brand logo and static assets
- src/components/layout/ - Layout.jsx (root authenticated shell), Sidebar.jsx (collapsible navigation drawer with role-filtered links), Topbar.jsx (header with hamburger, page title, theme toggle)
- src/components/ui/ - Shared UI components: Badge, Button, Card, ConfirmDialog, Input, Modal
- src/context/ - Five context providers: AuthContext, EmployeeContext, InvoiceContext, PayrollContext, ThemeContext
- src/data/ - Mock data files: mockEmployees.js, mockInvoices.js, mockPayroll.js
- src/features/auth/ - LoginPage.jsx

- src/features/dashboard/ - DashboardPage.jsx (branches by role)
- src/features/employees/ - EmployeesPage.jsx (employee directory, profile view, add/edit)
- src/features/invoices/ - InvoicesPage.jsx (invoice CRUD, status management, PDF export)
- src/features/payroll/ - PayrollPage.jsx (payroll processing, finalisation, payslip viewer)
- src/features/salary/ - SalaryPage.jsx (salary structure editor, report PDF export)
- src/router/ - AppRouter.jsx with ProtectedRoute, GuestRoute, and RoleRoute guards
- src/utils/ - calculations.js, formatters.js (LKR locale), pdfGenerator.js

Root configuration files include index.html, package.json, tailwind.config.js, postcss.config.js, vite.config.js, and .eslintrc. The entry point is src/main.jsx which renders the root App.jsx containing all context providers composed together.

12. Setup and Running the Project

12.1 Prerequisites

- Node.js version 18 or higher
- npm version 9 or higher

12.2 Installation and Development

Navigate to the project directory (cd fms-hsclogic) and run npm install to install all dependencies. Start the development server with npm run dev, which opens at http://localhost:5173 with hot module replacement. Changes to any source file reflect in the browser instantly.

12.3 Production Build

Run npm run build to generate a production build in the dist/ folder. The build completes with zero errors. A chunk size warning may appear for the jsPDF bundle - this is expected and non-blocking. Use npm run preview to serve the production build locally for final verification. Run npm run lint for code quality checks.

13. Known Limitations and Future Enhancements

13.1 Current Limitations

The following limitations are expected given the assignment scope, which explicitly limits implementation to the frontend/UI layer. These are not deficiencies but intentional scope boundaries that would be addressed in Phase 2 backend development.

Area	Detail
No backend or database	All data is in-memory and resets on page refresh (session is the only persistent state)
No real authentication security	Credentials are stored in a mock array in plain text - not suitable for production
No email or notification system	No automated reminders for overdue invoices or upcoming payroll dates
Single currency	All monetary values are in Sri Lankan Rupee (LKR) only
No audit log	Record changes are not tracked with timestamps or user attribution
Payroll filter limitation	History is filterable by year only - no search by employee name within payroll records

13.2 Recommended Future Enhancements

- Backend Integration: A REST API or GraphQL backend with a relational database (PostgreSQL recommended, as suggested in the FRS) and JWT-based authentication with token refresh
- Data Persistence: Replace in-memory mock state with real CRUD operations backed by a database, with optimistic UI updates and error handling
- Email Notifications: Automated overdue invoice reminders to clients, payslip distribution to employees on payroll finalisation, and payroll due-date alerts to admins
- Tax Calculations: APIT (Advance Personal Income Tax) computation per employee based on Sri Lanka's annual income tax brackets
- Leave Management: Integration with an approved leave record to prorate net pay for employees with unpaid leave days
- Partial Invoice Payments: Record partial payments with payment date and remaining balance tracking
- Audit Trail: Immutable log of all create, update, and delete operations with actor identity, timestamp, and before/after values
- Multi-Format Exports: CSV and Excel (XLSX) export options in addition to PDF for payroll and salary reports

- Multi-Company Support: Allow a single deployment to manage multiple entities under one admin account with entity-level data isolation

14. Conclusion

The HSCLogic Finance Management System has been successfully developed as a complete frontend implementation fulfilling all requirements defined in the SRS and FRS provided by HSClogic for the intern technical assignment. The system delivers a full financial operations platform covering the invoice lifecycle, employee record management, salary structure configuration, and monthly payroll processing. The FRS acceptance criteria - fully functional invoice CRUD with calculation, working salary module with payroll and history, proper validation, clean responsive UI, and structured production-ready code - have all been met.

The architecture is deliberately frontend-only as specified in the SRS constraints, making it straightforward to demonstrate all functional requirements within the two-week assignment timeline without the overhead of server infrastructure. All state management, business logic, validation rules, and PDF generation run entirely in the browser using React Context API, jsPDF, and Recharts. This approach allows the full system to be evaluated with a single npm install and npm run dev command. The implementation also extends beyond the minimum requirements with additional features including automatic overdue invoice detection, invoice duplication, bulk PDF export, dual-view employee browsing (table/card), CSV export, light/dark theme toggle, and a role-branched dashboard with interactive charts.

With the solid technical foundation now in place, the system is well-positioned for the Phase 2 enhancements outlined in Section 13.2 - particularly backend integration with PostgreSQL (as suggested in FRS, JWT-based authentication, APIT tax calculations for Sri Lanka, and automated email notifications - which would extend the system into a production-ready deployment capable of supporting HSCLogic's growing financial operations.